

REVISÃO E CORREÇÃO DO PROJETO SMARTCOFFEE

Banco de Dados • Consolidação das Aulas 01 a 07 • Regras de Negócio • MER/DER • Cardinalidades • Dicionário • DDL • CONSTRAINTS

Abertura

“Corrigir um banco de dados não é apenas corrigir SQL. É verificar se o banco representa corretamente o negócio.”

Pergunta norteadora: O modelo, a documentação e o SQL do SmartCoffee contam a mesma história?

Objetivos

- retomar os conceitos essenciais das Aulas 01 a 07;
 - compreender, registrar e validar regras de negócio do SmartCoffee;
 - realizar uma auditoria técnica no SmartCoffee;
 - corrigir entidades, atributos, PKs, FKs, relacionamentos e cardinalidades;
 - comparar DER, dicionário de dados e SQL;
 - aprofundar o uso de **CONSTRAINT** na modelagem e na implementação;
 - justificar tecnicamente cada correção realizada.
-

1. A sequência que será revisada

Etapa	Pergunta	Resultado esperado
1. Contexto	O que a cafeteria precisa controlar?	Regras de negócio
2. Entidades	Quais objetos precisam existir?	MER
3. Atributos	Que informações pertencem a cada objeto?	Entidades coerentes
4. Relacionamentos	Como as entidades se conectam?	Relacionamentos com verbos
5. Cardinalidades	Quantos podem/devem participar?	DER refinado
6. Chaves	Como identificar e relacionar registros?	PKs e FKs
7. Documentação	O significado dos campos está claro?	Dicionário de dados
8. Implementação	O SQL implementa o que foi modelado?	DDL + constraints

2. Revisão relâmpago

Entidade

Objeto relevante do domínio, como CLIENTE, PEDIDO e PRODUTO.

Atributo

Característica de uma entidade, como nome, preço e data.

Relacionamento

Associação entre entidades, normalmente expressa por verbo.

Cardinalidade

Quantidade mínima e máxima de ocorrências que podem participar.

Primary Key

Identifica unicamente cada registro.

Foreign Key

Referencia outra tabela e materializa um relacionamento.

3. Regras de Negócio — o ponto de partida do SmartCoffee

Importante: as aulas anteriores utilizam regras pontuais para explicar modelagem, relacionamentos e cardinalidades, mas ainda não definem um catálogo completo de regras do SmartCoffee. O modelo abaixo é uma **proposta didática** para orientar a revisão. Em um projeto real, cada regra deveria ser validada com o cliente ou responsável pelo processo.

3.1 O que é uma Regra de Negócio?

Uma **regra de negócio** é uma declaração que define uma condição, obrigação, limite, política ou comportamento que deve ser respeitado pelo sistema porque faz parte do funcionamento do negócio.

Não é apenas uma tela

“O sistema terá uma tela de clientes” é uma decisão de solução/interface.

É uma regra do domínio

“O CPF de um cliente cadastrado não pode se repetir” é uma regra que precisa permanecer verdadeira independentemente da tela utilizada.

3.2 Regra de negócio × requisito funcional

Tipo	Exemplo	O que define?
Regra de negócio	Um produto deve pertencer a uma categoria.	Uma condição do domínio.
Requisito funcional	O sistema deve permitir cadastrar produtos.	Uma função que o sistema oferecerá.
Decisão técnica	A tabela PRODUTO terá uma FK id_categoria.	Como a regra será implementada.

Fluxo esperado: Regra do negócio → Modelagem → Dicionário de Dados → Constraint/SQL.

3.3 Características de uma boa regra

- **Clara:** outra pessoa consegue entender sem adivinhar.
- **Atômica:** trata preferencialmente uma regra por vez.
- **Verificável:** é possível dizer se está sendo cumprida ou violada.
- **Independente de tecnologia:** deve existir antes de decidir MySQL, JavaScript ou HTML.
- **Rastreável:** deve possuir código, origem e ligação com elementos do modelo.
- **Consistente:** não pode contradizer outras regras do projeto.

3.4 Modelo para registrar uma Regra de Negócio

Campo	Como preencher
Código	Identificador único: RN-01, RN-02...
Título	Nome curto da regra.
Descrição	Regra escrita em linguagem clara.
Entidades envolvidas	CLIENTE, PEDIDO, PRODUTO etc.
Origem	Cliente, professor, processo observado, requisito do projeto.
Reflexo na modelagem	Cardinalidade, atributo obrigatório, entidade associativa etc.
Reflexo no banco	PK, FK, UNIQUE, NOT NULL, CHECK, DEFAULT ou regra que exige lógica adicional.
Exceções/observações	Condições especiais que precisam ser discutidas.

Exemplo preenchido

Código	RN-05
Título	Categoria obrigatória do produto
Descrição	Todo produto cadastrado deve pertencer a uma única categoria.
Entidades	PRODUTO e CATEGORIA
Reflexo no DER	CATEGORIA 1:N PRODUTO; PRODUTO participa obrigatoriamente.
Reflexo no SQL	id_categoria INT NOT NULL + FOREIGN KEY.
Exceção	Se o negócio permitir produto “sem categoria”, a regra deve ser alterada e a participação passa a ser opcional.

3.5 Catálogo proposto de Regras de Negócio — SmartCoffee

As regras abaixo formam uma **baseline didática**. A turma pode aceitar, alterar ou rejeitar cada regra, desde que registre a decisão e mantenha DER, dicionário e SQL coerentes.

Código	Regra proposta	Impacto principal
RN-01	Todo cliente cadastrado deve possuir um identificador único.	PK em CLIENTE.
RN-02	O nome do cliente é obrigatório.	nome NOT NULL.
RN-03	O CPF, quando adotado como obrigatório no projeto, não pode se repetir entre clientes.	CHAR(11), NOT NULL e UNIQUE.
RN-04	Um cliente pode estar cadastrado mesmo sem ter realizado pedidos.	CLIENTE participa de PEDIDO como (0,N).
RN-05	Todo pedido deve estar associado a um cliente cadastrado.	CLIENTE 1:N PEDIDO; FK obrigatória em PEDIDO.
RN-06	Todo produto deve pertencer a uma única categoria.	CATEGORIA 1:N PRODUTO; FK id_categoria NOT NULL.
RN-07	Uma categoria pode existir mesmo sem possuir produtos.	CATEGORIA participa como (0,N).
RN-08	O preço de venda de um produto não pode ser negativo.	DECIMAL + CHECK(preço >= 0).
RN-09	Novos produtos são considerados ativos por padrão, salvo decisão contrária no cadastro.	DEFAULT TRUE.
RN-10	Um pedido pode possuir vários produtos e um produto pode aparecer em vários pedidos.	N:N resolvido por ITEM_PEDIDO.
RN-11	Cada item do pedido deve referenciar exatamente um	Duas FKs obrigatórias em ITEM_PEDIDO.

Código	Regra proposta	Impacto principal
	pedido e um produto.	
RN-12	A quantidade de um item do pedido deve ser maior que zero.	CHECK(quantidade > 0).
RN-13	O preço praticado no momento da venda deve ser registrado no item do pedido.	preco_unitario em ITEM_PEDIDO; preserva histórico.
RN-14	Um fornecedor pode fornecer vários produtos e um produto pode possuir vários fornecedores.	N:N resolvido por FORNECEDOR_PRODUTO.
RN-15	O mesmo fornecedor não deve ser vinculado duas vezes ao mesmo produto.	PK composta ou UNIQUE(id_fornecedor,id_produto).
RN-16	No modelo simplificado, cada produto possui no máximo um registro atual de estoque.	PRODUTO 1:1 ESTOQUE com FK UNIQUE.
RN-17	A quantidade em estoque não pode ser negativa.	CHECK(quantidade >= 0).
RN-18	Uma reserva deve estar vinculada a uma mesa válida.	MESA 1:N RESERVA; FK em RESERVA.
RN-19	Uma mesa pode existir sem reservas cadastradas.	MESA participa como (0,N).
RN-20	O status de um pedido deve pertencer ao conjunto de estados definido pelo projeto.	Domínio documentado e CHECK ou solução equivalente.

3.6 Regras que exigem discussão — não há uma única resposta automática

Pedido sem cliente: o SmartCoffee permitirá venda anônima no balcão? Se sim, RN-05 precisa mudar e `id_cliente` poderá ser opcional.

Estoque: será um único saldo por produto ou haverá movimentações, lotes e locais de estoque? A resposta pode alterar completamente a entidade ESTOQUE.

Pagamento: um pedido poderá ter vários pagamentos/formas de pagamento? Se sim, a cardinalidade precisa representar isso.

Reserva: como impedir duas reservas para a mesma mesa no mesmo intervalo? Essa regra não é resolvida apenas por uma FK simples e pode exigir validação adicional.

3.7 Nem toda regra vira uma CONSTRAINT simples

Algumas regras são fáceis de implementar diretamente no banco, como unicidade de CPF ou preço não negativo. Outras dependem de mais de uma linha, de horário, de estado do processo ou de uma sequência de operações.

Regra	Implementação provável
CPF não pode repetir	UNIQUE.
Preço não pode ser negativo	CHECK.
Pedido aponta para cliente existente	FOREIGN KEY.
Pedido deve terminar com pelo menos um item	Pode exigir transação, aplicação, procedure/trigger ou validação de processo; uma FK isolada não garante a regra completa.
Não permitir reservas de uma mesma mesa em horários sobrepostos	Normalmente exige lógica adicional para comparar intervalos.

3.8 Atividade rápida — da regra ao banco

Para cada regra, preencha as quatro etapas:

Regra	DER	Dicionário	SQL/ Constraint
CPF não pode repetir			
Produto pertence a uma categoria			
Quantidade do item deve ser positiva			
Fornecedor pode fornecer vários produtos			

4. Processo de correção do SmartCoffee

Passo 1 — Regra antes do SQL

Regra: “Um cliente pode realizar vários pedidos. Cada pedido deve pertencer a um cliente cadastrado.”

Essa frase já permite concluir que CLIENTE e PEDIDO são entidades, existe um relacionamento 1:N, PEDIDO está no lado N e precisa de uma FK obrigatória para CLIENTE.

Passo 2 — Revisar entidades e atributos

Entidades possíveis:

CLIENTE CATEGORIA PRODUTO PEDIDO ITEM_PEDIDO
PAGAMENTO FORNECEDOR ESTOQUE MESA RESERVA

Erro comum: transformar preço, data ou nome em entidade; ou tratar um verbo, como “realiza”, como entidade.

Passo 3 — Eliminar grupos repetitivos

Evite:

```
PEDIDO(id_pedido, produto1, produto2, produto3,  
        quantidade1, quantidade2, quantidade3)
```

O relacionamento PEDIDO × PRODUTO é N:N e deve ser resolvido por ITEM_PEDIDO.

Passo 4 — Ler a cardinalidade nos dois sentidos

Pergunte:

Para um A, quantos B podem existir?

Para um B, quantos A podem existir?

Relação	Resultado típico	Observação
CLIENTE — PEDIDO	1:N	FK id_cliente em PEDIDO
CATEGORIA — PRODUTO	1:N	FK id_categoria em PRODUTO
PEDIDO — PRODUTO	N:N	resolver com ITEM_PEDIDO
FORNECEDOR — PRODUTO	N:N	resolver com FORNECEDOR_PRODUTO

Passo 5 — Cardinalidade mínima

Notação	Significado
(0,1)	opcional, no máximo um
(1,1)	obrigatório e único
(0,N)	opcional, muitos possíveis
(1,N)	obrigatório, muitos possíveis

5. CONSTRAINT na modelagem

Constraint é uma restrição/regra que protege a consistência do modelo. Na modelagem ela pode aparecer como:

- **identificação:** toda entidade precisa ser identificável;
- **unicidade:** CPF não deve se repetir;

- **obrigatoriedade:** nome de produto deve existir;
- **integridade referencial:** PEDIDO só pode apontar para CLIENTE existente;
- **cardinalidade:** CATEGORIA 1:N PRODUTO;
- **domínio:** preço não pode ser negativo;
- **participação:** algumas relações podem ser opcionais e outras obrigatórias.

Importante: a constraint não nasce no comando SQL. Ela nasce na regra de negócio, aparece na modelagem, é registrada no dicionário e depois é implementada no SGBD.

Como registrar no Dicionário de Dados

Campo	Tipo	PK	FK	Nulo?	Regra / Constraint
id_cliente	INT	Sim	Não	Não	AUTO_INCREMENT
cpf	CHAR(11)	Não	Não	Não	UNIQUE
preco	DECIMAL(10,2)	Não	Não	Não	CHECK(preco >= 0)
id_categoria	INT	Não	Sim	Não	REFERENCES categoria

6. CONSTRAINT em SQL

Recurso	Objetivo
PRIMARY KEY	garantir identificação única
FOREIGN KEY	garantir integridade referencial
UNIQUE	impedir duplicidade
NOT NULL	exigir preenchimento
CHECK	validar uma condição
DEFAULT	definir valor inicial

Constraints nomeadas

```
CREATE TABLE cliente (  
    id_cliente INT AUTO_INCREMENT,  
    nome VARCHAR(100) NOT NULL,  
    cpf CHAR(11) NOT NULL,  
  
    CONSTRAINT pk_cliente  
        PRIMARY KEY (id_cliente),  
  
    CONSTRAINT uq_cliente_cpf  
        UNIQUE (cpf)  
);
```

Nomear constraints ajuda na manutenção, documentação e interpretação de erros.

FK nomeada

```
CONSTRAINT fk_pedido_cliente  
    FOREIGN KEY (id_cliente)  
    REFERENCES cliente(id_cliente)
```

CHECK

```
CONSTRAINT ck_produto_preco  
    CHECK (preco >= 0)
```

Em MySQL moderno, CHECK é validado pelo SGBD.

7. Relacionamentos e constraints

1:N

CLIENTE 1 ————— N PEDIDO

PEDIDO recebe id_cliente como FK.

1:1

PRODUTO 1 ————— 1 ESTOQUE

Uma FK única ajuda a garantir o 1:1:

```
CONSTRAINT uq_estoque_produto UNIQUE
(id_produto)
CONSTRAINT fk_estoque_produto
    FOREIGN KEY (id_produto) REFERENCES
produto(id_produto)
```

N:N

FORNECEDOR N — N PRODUTO

↓

FORNECEDOR 1 – N FORNECEDOR_PRODUTO N – 1
PRODUTO

```
CONSTRAINT pk_fornecedor_produto
    PRIMARY KEY (id_fornecedor, id_produto)
```

8. ON DELETE e ON UPDATE

Ação	Comportamento	Quando pensar nela
RESTRICT / NO ACTION	impede ação que quebraria referências	quando histórico deve ser preservado
CASCADE	propaga a ação aos dependentes	quando dependente não faz sentido sozinho
SET NULL	remove a referência e mantém o registro	quando a relação pode ficar opcional

```
CONSTRAINT fk_produto_categoria
  FOREIGN KEY (id_categoria)
  REFERENCES categoria(id_categoria)
  ON UPDATE CASCADE
  ON DELETE RESTRICT
```

Cuidado: CASCADE não deve ser usado por costume. A regra de negócio precisa justificar o comportamento.

9. Exemplo completo corrigido

```
CREATE TABLE produto (  
    id_produto INT AUTO_INCREMENT,  
    nome VARCHAR(100) NOT NULL,  
    descricao VARCHAR(255),  
    preco DECIMAL(10,2) NOT NULL,  
    ativo BOOLEAN NOT NULL DEFAULT TRUE,  
    id_categoria INT NOT NULL,  
  
    CONSTRAINT pk_produto PRIMARY KEY  
    (id_produto),  
    CONSTRAINT ck_produto_preco CHECK (preco >=  
0),  
    CONSTRAINT fk_produto_categoria  
        FOREIGN KEY (id_categoria)  
        REFERENCES categoria(id_categoria)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
);
```

10. Auditoria: DER × Dicionário × SQL

Modelagem:

```
CLIENTE (0,N) ——— PEDIDO (1,1)
```

SQL recebido:

```
CREATE TABLE pedido (  
    id_pedido INT PRIMARY KEY,  
    data_pedido DATETIME,  
    id_cliente INT  
);
```

Erro 1: existe relacionamento no DER, mas não existe FOREIGN KEY.

Erro 2: se todo pedido exige cliente, id_cliente não deveria aceitar NULL.

Erro 3: a data pode precisar ser obrigatória conforme a regra.

Versão coerente

```
CREATE TABLE pedido (  
    id_pedido INT AUTO_INCREMENT,  
    data_pedido DATETIME NOT NULL DEFAULT  
CURRENT_TIMESTAMP,  
    id_cliente INT NOT NULL,  
    CONSTRAINT pk_pedido PRIMARY KEY  
(id_pedido),  
    CONSTRAINT fk_pedido_cliente  
        FOREIGN KEY (id_cliente)  
        REFERENCES cliente(id_cliente)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
);
```

11. Roteiro de correção em sala

Fase A — Conceitual

1. cada grupo apresenta o que o SmartCoffee precisa controlar;
2. lê três regras de negócio;
3. a turma identifica entidades e relacionamentos;
4. o grupo compara com o próprio DER.

Fase B — DER

- todas as entidades fazem sentido?

- todo atributo está na entidade correta?
- toda entidade possui PK?
- cardinalidades foram lidas nos dois sentidos?
- há N:N ainda não resolvido?
- mínimo 0/1 está coerente com a obrigatoriedade?

Fase C — Dicionário

- tipos e tamanhos estão corretos?
- PK/FK estão documentadas?
- NULL/NOT NULL correspondem às regras?
- UNIQUE, DEFAULT e CHECK estão registrados?

Fase D — SQL

- o CREATE TABLE corresponde ao dicionário?
- PKs e FKs estão realmente implementadas?
- constraints importantes receberam nomes?
- ON DELETE/ON UPDATE foram escolhidos conscientemente?

12. Atividade prática — Auditoria Técnica SmartCoffee

Missão: revisar o projeto antes de ele ser liberado para desenvolvimento.

Entrega 1 — Mapa de inconsistências

Encontre pelo menos **8 problemas ou melhorias** e registre:

Problema	Por que é um problema?	Correção	Conceito

Entrega 2 — DER revisado

- mínimo de 8 entidades;
- PK em todas;
- mínimo de 5 relacionamentos;
- cardinalidade mínima e máxima;
- ao menos uma entidade associativa;
- FKs coerentes.

Entrega 3 — Dicionário revisado

Documento pelo menos 4 tabelas, incluindo regras e constraints.

Entrega 4 — SQL corrigido

- 4 PRIMARY KEY nomeadas;
- 3 FOREIGN KEY nomeadas;
- 2 UNIQUE;
- 2 CHECK;
- NOT NULL coerente;
- 1 DEFAULT;
- 1 decisão explícita de ON DELETE/ON UPDATE.

Entrega 5 — Defesa técnica

Escolha 3 decisões e explique: regra de negócio → DER → dicionário → SQL.

13. Desafio de correção

Corrija:

```
CREATE TABLE produto (  
    id INT,  
    nome VARCHAR(255),  
    preco VARCHAR(50),  
    categoria VARCHAR(100)  
);
```

Sua solução deve considerar PK, tipo monetário, CATEGORIA como entidade, FK, obrigatoriedade, CHECK, constraints nomeadas e cardinalidade.

14. Rubrica sugerida

Critério	Pontos
Regras de negócio e coerência conceitual	20
Entidades, atributos e chaves	15
Relacionamentos e cardinalidades	20
Dicionário de dados	15
DDL, tipos e constraints	20
Justificativa técnica e organização	10
Total	100

15. Perguntas relâmpago

1. Qual a diferença entre entidade e atributo?
2. O que uma PK garante?
3. O que uma FK representa?
4. Onde fica a FK no 1:N?
5. Como resolvemos N:N?
6. O que significa (0,N)?
7. Qual a relação entre obrigatoriedade e NOT NULL?

8. Por que usar DECIMAL para dinheiro?
 9. Qual a diferença entre UNIQUE e PRIMARY KEY?
 10. Para que serve CHECK?
 11. Por que nomear uma CONSTRAINT?
 12. Por que DER, dicionário e SQL precisam permanecer coerentes?
-

Encerramento

O banco correto não é o que apenas aceita um CREATE TABLE. É o que representa as regras do negócio e impede estados incoerentes.